

САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

Факультет прикладной математики-процессов управления

**Программа бакалавриата
“Большие данные и распределенная цифровая платформа”**

ОТЧЕТ

по лабораторной работе №4

по дисциплине «Алгоритмы и структуры данных»

**на тему «Исследование генетического алгоритма. Изучение различных
кодировок генотипа»**

Вариант – 1

**Студент гр. 23Б15-пу
Трофимов И. А.**

**Преподаватель
Дик А.Г.**

Санкт-Петербург

2024 г.

Оглавление

1. Цель работы	3
2. Теоретическая часть	3
3. Описание задачи	4
4. Основные шаги программы	4
5. Описание программы	6
6. Рекомендации пользователя	7
7. Рекомендации программиста	7
8. Исходный код программы:	7
9. Контрольный пример	8
10. Исследование	9
11. Вывод	10
12. Источники	10

Цель работы

Целью лабораторной работы является исследование двух основных способов кодирования генотипа хромосом в генетическом алгоритме и проверка их эффективности. Этот алгоритм должен решить задачу нахождения минимума функции.

Теоретическая часть

Генетический алгоритм (ГА) — это метод оптимизации, вдохновленный механизмами естественного отбора и эволюции. Основная цель ГА — найти оптимальное решение для заданной функции путем эволюции популяции возможных решений. Алгоритм работает следующим образом:

1. **Инициализация популяции:** Создается начальная популяция из случайных решений (хромосом).
2. **Оценка пригодности:** Каждый индивид (хромосома) оценивается с помощью целевой функции, которая определяет его пригодность. Чем лучше значение функции, тем более "приспособленным" считается индивид.
3. **Селекция:** Из популяции выбираются индивиды для размножения. Обычно используются методы, как турнирный отбор, где случайные пары сравниваются, и более приспособленный индивид проходит в следующий этап.
4. **Кроссинговер:** Выбранные индивиды "скрещиваются" для создания потомков. Происходит обмен генами (параметрами) между родителями с некоторой вероятностью, что способствует разнообразию решений.
5. **Мутация:** Случайным образом изменяются некоторые гены потомков с небольшой вероятностью, чтобы избежать преждевременной сходимости и улучшить исследование пространства решений.

6. **Обновление популяции:** Потомки заменяют предыдущую популяцию, и процесс повторяется для заданного числа поколений.
7. **Выходное условие:** Алгоритм останавливается, когда достигается заданное число поколений или когда изменения в значениях становятся незначительными.

Также в алгоритме была выполнена модификация в части создания новой популяции посредством элитарного отбора. **Элитарный отбор** — это модификация генетического алгоритма, направленная на сохранение наиболее приспособленных индивидов на протяжении всех поколений. Данная техника обеспечивает перенос наилучших решений, найденных на каждом этапе, в следующее поколение без изменений. Основная цель элитарного отбора — улучшить эффективность и результативность генетического алгоритма, минимизируя потерю уже найденных хороших решений из-за случайного кроссовера или мутаций.

Описание задачи

Задача состоит в нахождении глобального минимума функции с помощью ГА. Алгоритм направлен на нахождение такой пары значений (x, y) , при которой значение функции: $f(x,y)=x^2+3y^2+2xy$ становится минимально возможным. Глобальный минимум для этой функции находится в точке $(0,0)$, где значение функции равно 0.

Основные шаги программы

1. **Инициализация популяции:** создается начальная популяция случайных особей (пар значений x и y).
2. **Селекция:** отбор лучших особей из текущего поколения для создания следующего поколения, основываясь на их «приспособленности».

3. **Кроссинговер**: скрещивание выбранных пар особей для генерации потомков с некоторыми признаками обоих родителей.
4. **Мутация**: случайное изменение значений потомков для поддержания генетического разнообразия и предотвращения преждевременной сходимости.
5. **Элитарный отбор**: добавление в следующее поколение лучших особей из текущего поколения для гарантии, что оптимальные решения не потеряются.
6. **Оценка результатов**: на каждом этапе вычисляется значение функции для лучшей особи текущего поколения, чтобы отслеживать прогресс к оптимальному решению.

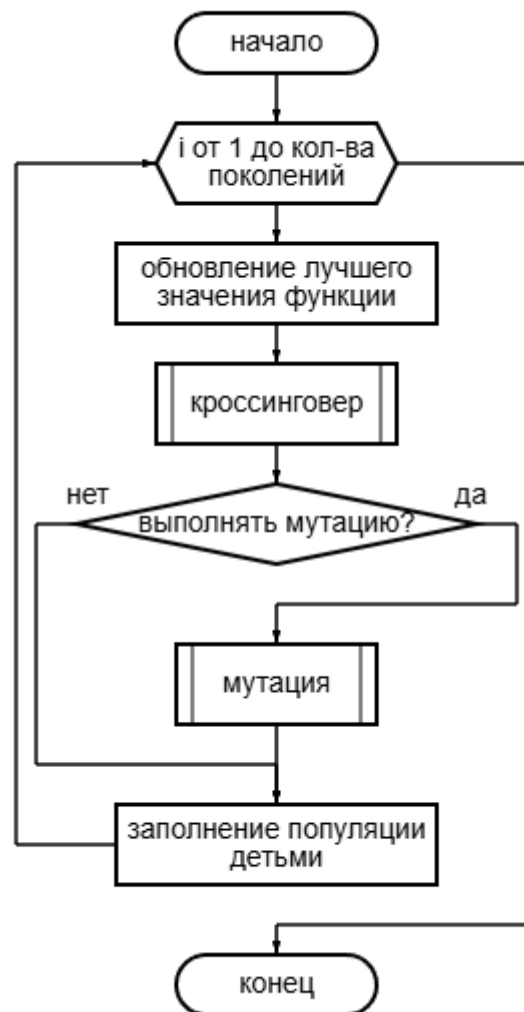


Рис 1. Блок-схема основной программы



Блок-схемы подпрограмм

Рис 2.

Описание программы

Программная реализация написана на языке Python 3.12.2 с использованием библиотеки `numpy`[\[1\]](#) и встроенного модуля `tkinter`[\[2\]](#). В процессе разработки программы использовался следующий модуль:

Таблица1 main.py

Функция	Описание	Возвращаемое значение
objective_function()	Получает значение исходной функции	float
crossover()	Скрещивает особи и создает кроссинговер	tuple
mutate()	Создает мутацию в генах индивида	tuple
run_generation()	Главная функция, включающая в себя все остальные. Реализует ГА	None
update_table()	Обновляет график	None
draw_population()	Отрисовывает точки	None
run_step()	Запуск ГА на кол-во поколений	None

Рекомендации пользователя

Для запуска программы убедитесь, что у вас установлен Python и необходимые библиотеки, такие как numpy [\[1\]](#). Код можно запустить в среде разработки или через командную строку, используя консоль для настройки параметров и генерации данных. Запуск программы производится через файл main.py.

При запуске программы вам будет предложено выбрать параметры запускаемого алгоритма. Вводите ответы соответствующие поля в GUI. Результаты работы будут представлены в таблице.

Рекомендации программиста

Для поддержания актуальности и работоспособности программы используйте последние версии библиотек, особенно numpy [\[1\]](#). Применяйте практики

надлежащего именования переменных и функций для улучшения читаемости кода.

Исходный код программы:

[Labs/4 at commt-5 · hanglider/Labs](#)

Контрольный пример

1. Запуск программы

Для запуска программы используйте файл `main.py`. Программа загружает GUI и позволяет пользователю настроить параметры ГА.

2. Выбор параметров ГА

После запуска программы пользователю будет предложено выбрать параметры (Рис. 3).

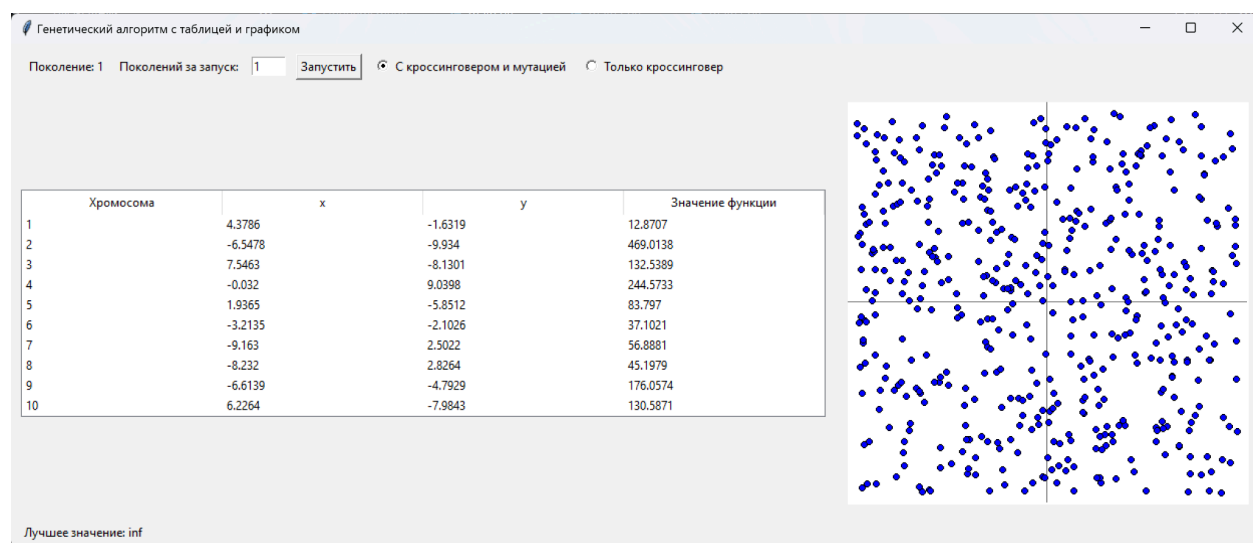


Рис 3. Пример выбора параметров

3. Обработка данных и вывод результатов

После выбора параметров программа запускает алгоритм, а затем выводит результаты в таблице (Рис. 4).

	Хромосома	x	y	Значение функции
1		-0.1891	0.0582	0.0239
2		-0.9966	0.414	0.6823
3		0.3056	-0.5815	0.7524
4		-0.3741	0.8485	1.6648
5		1.2786	0.0179	1.6814
6		-1.3678	0.8972	1.8315
7		1.352	-0.9289	1.9047
8		-1.8731	0.601	2.3407
9		1.876	-0.7779	2.4161
10		-1.9072	0.7573	2.4692

Лучшее значение: 0.0239

Рис 4. Результат работы алгоритма

Исследование

В рамках данной лабораторной работы было проведено сравнение работы алгоритма с модификацией или без. ГА – эвристический алгоритм, поэтому 100% сходимость гарантировать нельзя. На основе проведенных тестов можно сделать вывод, что модификация значительно улучшает сходимость алгоритма, а модифицированный алгоритм быстрее приходит к наиболее точному ответу. Модификация оператора кроссинговера позволяет улучшить генетический алгоритм за счет более эффективного поиска в пространстве решений. Усиление кроссинговера помогает сохранить и передавать полезные комбинации признаков между поколениями, что ускоряет достижение оптимального решения и повышает качество итогового результата.

Вывод

В рамках данной работы был разработан алгоритм для нахождения глобального минимума функции. Программа была протестирована на функции $x^2 + 3y^2 + 2xy$. Реализованный алгоритм обеспечивает возможность оптимизации различных функций, например поиск экстремумов или решение NP полных задач. В части создания новой популяции была

реализована модификация с помощью элитарного отбора, что помогло сделать каждое новое поколение не хуже предыдущего.

Источники

1. Numpy documentation // Numpy URL: <https://numpy.org> (дата обращения: 27.10.2024).
2. Tkinter documentation // Tkinter URL: <https://docs.python.org/3/library/tkinter.html> (дата обращения: 28.10.2024)